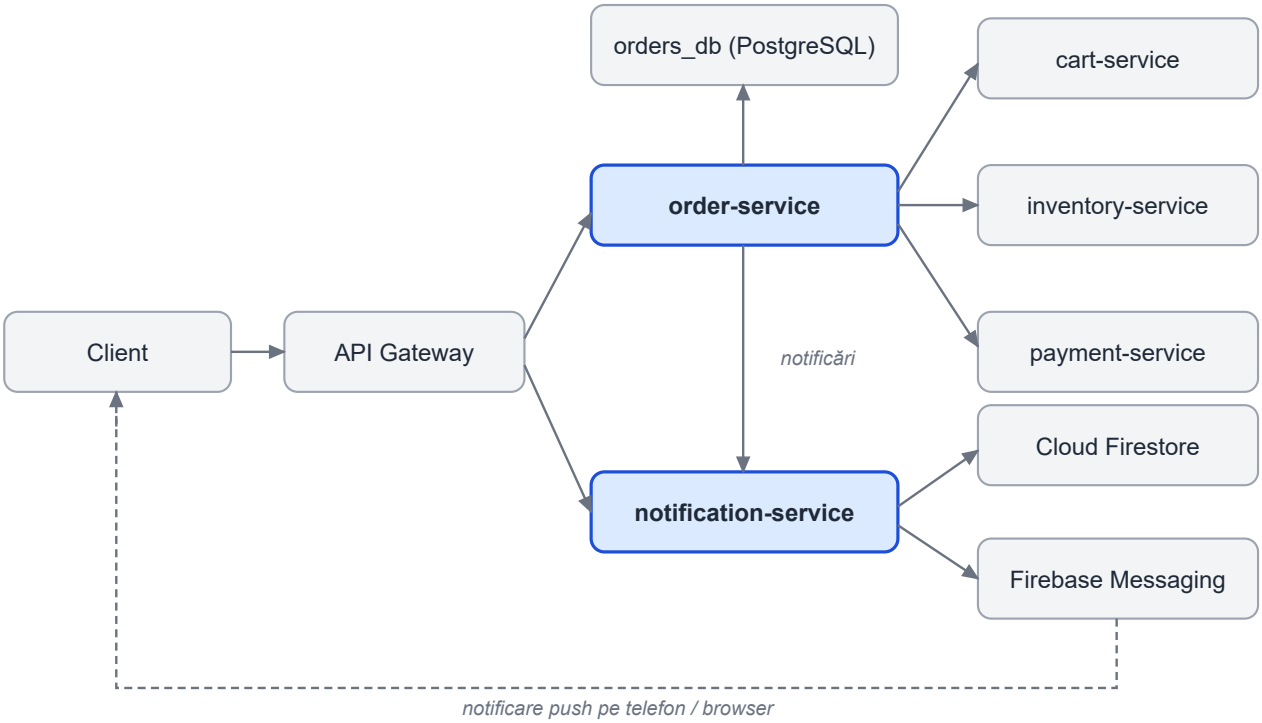
Order Service & Notification Service

Structura arhitecturii și ce mai trebuie adăugat · 28.10.2026

1. Pe scurt

- **order-service**: plasează comenzile (checkout), le ține statusul și oferă istoricul comenzilor. Datele stau în PostgreSQL (*orders_db*).
- **notification-service**: trimite notificări push prin Firebase Cloud Messaging și le salvează în Cloud Firestore, ca să apară și în lista de notificări din aplicație.

2. Diagrama



Cu albastru sunt cele două servicii. Față de diagrama din README apar două legături noi: **order-service → cart-service** (prețurile se iau din coș, nu de la client, ca să nu poată fi modificate din frontend) și **Cloud Firestore** pentru notification-service.

3. Cum funcționează checkout-ul

1. order-service citește coșul din **cart-service** (produse + prețuri).
2. Salvează comanda cu statusul **PENDING**.
3. Cere **inventory-service** să rezerve stocul. Dacă nu mai e stoc, comanda devine **REJECTED**.
4. Cere **payment-service** să facă plata.
5. Plata reușește: rezervarea se confirmă, comanda devine **CONFIRMED** și coșul se golește. Plata eșuează: stocul se eliberează și comanda devine **PAYMENT_FAILED**.
6. Trimite notificarea către **notification-service**.

Statusuri: PENDING → STOCK_RESERVED → CONFIRMED → SHIPPED → DELIVERED. Pe lângă ele: REJECTED, PAYMENT_FAILED, CANCELLED.

4. Unde se stochează datele

Serviciu	Unde	Ce conține
order	PostgreSQL · orders	userId, status, total, adresa de livrare, id rezervare, id plată
order	PostgreSQL · order_items	produs, vânzător, nume, preț, cantitate (copie de la momentul comenzii)
notification	Firestore · notifications	userId, tip, titlu, text, citită / necitită, status trimitere
notification	Firestore · device_tokens	token-ul FCM al fiecărui device, userId, platformă (web / android / ios)

5. Endpoint-uri

Metodă	Cale	Ce face
POST	`/api/orders/checkout`	plasează comanda din coș
GET	`/api/orders`	istoricul comenzilor
GET	`/api/orders/{id}`	detaliile unei comenzi
POST	`/api/orders/{id}/cancel`	anulează comanda
PATCH	`/api/orders/{id}/status`	vânzătorul marchează SHIPPED / DELIVERED
GET	`/api/notifications`	notificările mele
PATCH	`/api/notifications/{id}/read`	marchează notificarea ca citită
POST	`/api/notifications/devices`	frontend-ul înregistrează token-ul FCM după login
POST	`/internal/notifications`	apelat doar de order-service, nu e expus în gateway

6. Ce am nevoie de la voi

Cine	Ce trebuie să ofere
api-gateway	Validează token-ul Firebase și trimite mai departe header-ele **X-User-Id** și **X-User-Role**. Nu expune rutele **/internal/****.
cart-service	**GET /internal/carts/{userId}**: produsele din coș (productId, sellerId, nume, preț, cantitate). **DELETE /internal/carts/{userId}**: golește coșul.
inventory-service	**POST /internal/inventory/reservations**: rezervă stocul atomic, răspunde **409** dacă nu e stoc. Plus **confirm** și **eliberare** rezervare.
payment-service	**POST /internal/payments** cu {orderId, userId, amount, currency}; răspunde cu {paymentId, status: SUCCEEDED / FAILED}.
frontend	Cere permisiunea de notificări, ia token-ul FCM și îl trimite la **/api/notifications/devices**.

Căile sunt o propunere. Dacă vreți alte nume sau alte câmpuri, spuneți-mi și le schimb.

7. Ce mai trebuie adăugat

order-service

- refund și repunerea stocului la anularea unei comenzi deja plătite
- un job care anulează comenzile rămase blocate (PENDING / STOCK_RESERVED)
- endpoint pentru vânzător: comenzile cu produsele lui (încasări)
- teste pentru fluxul de checkout

notification-service

- retrimitere automată pentru notificările care au eșuat
- numărul de notificări necitite (pentru badge)
- teste

Comun

- rutele în api-gateway, intrările în docker-compose și colecția Postman
- cheia Firebase (service account) nu se urcă pe git, o trimit separat